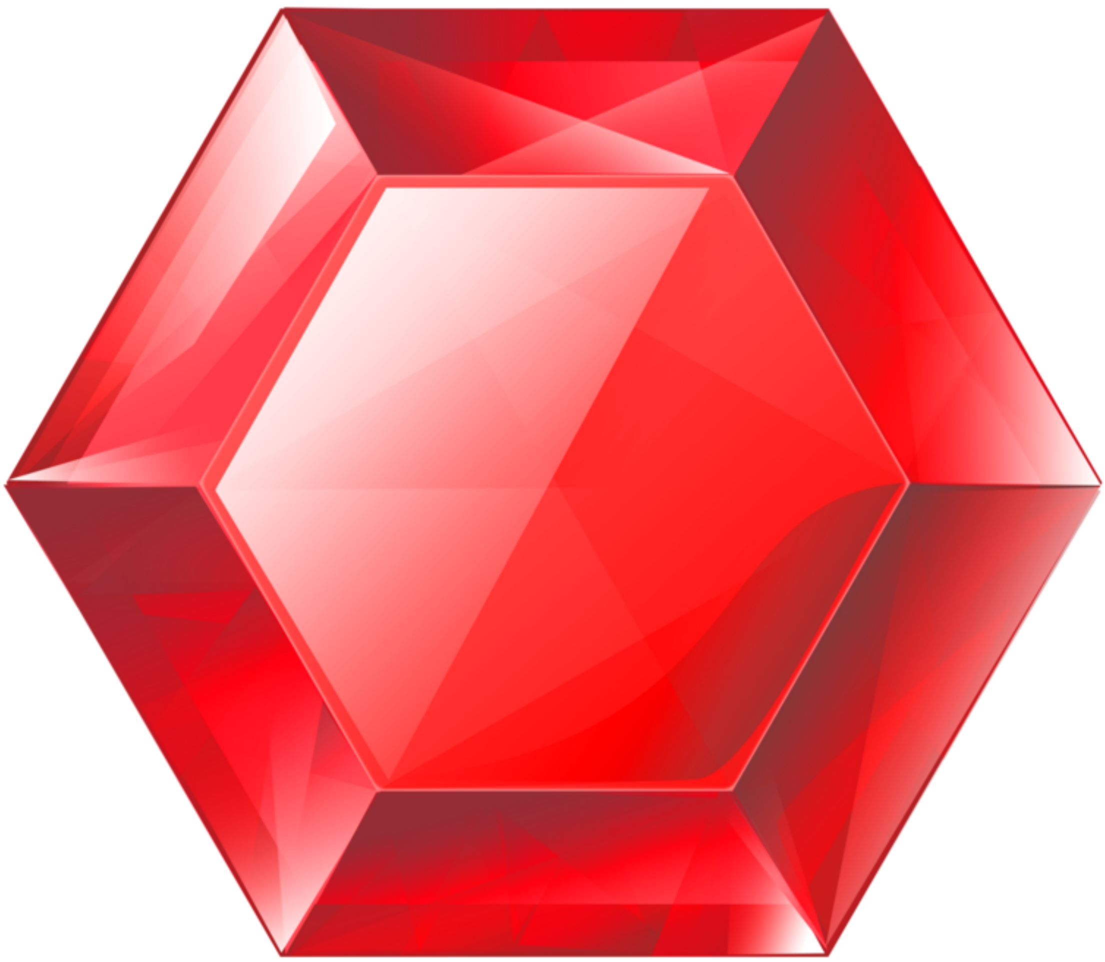
# gemstone-rs Codegen Guide

## Generating checked-in Rust wrappers for GemStone classes



Generated from the Markdown sources in gemstone-rs/docs.

# Contents

# Rust Codegen

`gemstone-rs` codegen creates small Rust wrapper structs around GemStone OOPs. The goal is to move repeated selector strings into checked-in generated code while keeping the mapping reviewable.

## Install

```
cargo install gemstone-rs-cli
gemstone-rs codegen --help
```

From a checkout:

```
cargo run -p gemstone-rs-cli -- codegen preview examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen explain examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen explain --json examples/codegen/gemstone-
rs.codegen
cargo run -p gemstone-rs-cli -- codegen check-profile default examples/codegen/
gemstone-rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- codegen explain-profile --json default examples/
codegen/gemstone-rs.codegen-profiles.json
```

## Config Format

The config is intentionally line-oriented:

```
output = examples/codegen/generated/gemstone_wrappers.rs
class = Object
method = Object>>printString | return=String | doc=Return the receiver printString.
method = Object>>class
```

Use `Dictionary:ClassName` when a class should resolve from a specific dictionary:

```
class = UserGlobals:OkzBooking
method = UserGlobals:OkzBooking>>findById: | args=id | return=Oop | doc=Find a
booking by id.
```

Class-side references use `class`:

```
class = UserGlobals:OkzBooking class
method = UserGlobals:OkzBooking class>>findById: | args=id | return=Oop
```

## Method Metadata

| Option | Example | Purpose |
| --- | --- | --- |
| `args` | `args=id,user` | Names generated Rust function arguments. |
| `return` | `return=String` | Generates a typed return helper instead of `Value`. |
| `doc` | `doc=Find by id.` | Writes a Rust doc comment above the generated method. |

When `args` is omitted, codegen now infers useful argument names from selector keywords. For example `at:put:` becomes `at, put`, and `withCustomer:amount:` becomes `customer, amount`. Provide explicit `args=...` when the selector keywords are not good Rust argument names.

Generated wrapper files include a small `#[cfg(test)]` surface-name test stub, so downstream crates can keep generated wrapper names visible to Rust test tooling. Use `codegen explain` before generating when you want a readable summary of the output file, test stub, wrapper classes, selectors, argument names, return helpers, mapped structs, and BridgeRoot field mappings:

```
gemstone-rs codegen explain examples/codegen/gemstone-rs.codegen
gemstone-rs codegen explain --json examples/codegen/gemstone-rs.codegen
gemstone-rs --env-file .env.gemstone-rs codegen check examples/codegen/gemstone-
rs.codegen
```

The JSON form is intended for editor and explorer integrations that want to render the output path, generated test stubs, class wrappers, selector arguments, return helpers, and mapped fields as structured data.

Machine-readable schema files are committed for tooling:

```
schemas/gemstone-rs.codegen.schema.json
schemas/gemstone-rs.codegen-explain.schema.json
schemas/gemstone-rs.codegen-profiles.schema.json
schemas/gemstone-rs.profile-check.schema.json
```

`gemstone-rs.codegen` remains the line-oriented CLI format. The config schema describes an equivalent structured JSON model for editor panels and generated summaries, while the explain schema matches `codegen explain --json`. The profile check schema matches `profile check --json` and the explorer `/api/codegen/profiles/check` endpoint.

Supported return types:

| Config value | Rust return |
|---|---|
| `Value` | `gemstone_rs::Value` |
| `String` | `String` |
| `SmallInt` | `i64` |
| `Bool` | `bool` |
| `Oop` | `gemstone_rs::Oop` |

## Mapped Structs

Codegen can also emit typed `BridgeMapped` structs for values stored under `BridgeRoot`:

```
mapped = BookingDraft | doc=A typed Rust payload stored under BridgeRoot.
field = BookingDraft.name | type=String | key=name
field = BookingDraft.amount | type=SmallInt | key=amount | key_type=Symbol
field = BookingDraft.currency | type=String | key=currency
field = BookingDraft.tags | type=Vec<String> | key=tags
field = BookingDraft.labels | type=BTreeMap<String, String> | key=labels |
doc=String-keyed labels.
field = BookingDraft.note | type=Option<String> | key=note | doc=Optional note.
```

Supported field types are `String`, `SmallInt`, `Bool`, `Oop`, `Mapped<OtherStruct>`, `Vec<T>`, `BTreeMap<String, T>`, and `Option<T>`. `Map<String, T>` is accepted as a shorter alias, and `Dictionary<T>` is an alias for `BTreeMap<String, T>`. Map fields store string-keyed relationship metadata as a GemStone `Dictionary`. Optional fields write `None` as GemStone `nil`; read-back returns `None` when the key is missing or when the stored value is `nil`.

Field options:

| Option | Example | Purpose |
|---|---|---|
| `type` | `type=Option<String>` | Rust/GemStone field conversion. |
| `key` | `key=amount` | GemStone dictionary key. |
| `key_type` | `key_type=Symbol` | Use string keys or symbol keys explicitly. |
| `doc` | `doc=Booking amount.` | Writes a Rust doc comment above the field. |

## Commands

Create a starter config:

```
gemstone-rs codegen init gemstone-rs.codegen
```

Generate a config from a live stone:

```
gemstone-rs codegen discover gemstone-rs.codegen Object
gemstone-rs codegen discover gemstone-rs.codegen UserGlobals:OkzBooking
gemstone-rs codegen discover-mapping gemstone-rs.codegen BookingDraft Object
```

From a checkout:

```
cargo run -p gemstone-rs --example codegen_discover
cargo run -p gemstone-rs --example codegen_discover_mapping
```

From an installed CLI, scaffold standalone projects:

```
gemstone-rs examples scaffold codegen_discover ./gemstone-rs-codegen-discover
gemstone-rs examples scaffold codegen_discover_mapping ./gemstone-rs-codegen-
discover-mapping
gemstone-rs examples scaffold profile_codegen_workflow ./gemstone-rs-profile-codegen
```

`profile_codegen_workflow` writes `gemstone-rs.codegen` and `gemstone-rs.codegen-profiles.json` into the generated project, then the Rust program runs explain, generate, and profile check against those files.

Preview without writing:

```
gemstone-rs codegen preview gemstone-rs.codegen
```

Show a generated diff:

```
gemstone-rs codegen diff gemstone-rs.codegen
```

Check freshness in CI:

```
gemstone-rs codegen check gemstone-rs.codegen
cargo test --manifest-path examples/codegen-wrapper-check/Cargo.toml
```

Write generated wrappers:

```
gemstone-rs codegen generate gemstone-rs.codegen
```

## Explorer Profiles

The local explorer can save repeatable codegen workflows as profiles. A profile captures:

- codegen root
- config path
- mapped Rust type
- GemStone class

Use the browser UI to save a local profile, export a single profile as JSON, or load a project-level profile file. The repository includes a sample:

```
examples/codegen/gemstone-rs.codegen-profiles.json
```

Validate it from CI or a terminal:

```
gemstone-rs profile sample
gemstone-rs profile init gemstone-rs.codegen-profiles.json
gemstone-rs profile validate gemstone-rs.codegen-profiles.json
gemstone-rs profile validate --json gemstone-rs.codegen-profiles.json
gemstone-rs profile list gemstone-rs.codegen-profiles.json
gemstone-rs profile show default gemstone-rs.codegen-profiles.json
gemstone-rs profile resolve default gemstone-rs.codegen-profiles.json
gemstone-rs profile check gemstone-rs.codegen-profiles.json
gemstone-rs profile check --json gemstone-rs.codegen-profiles.json
gemstone-rs codegen preview-profile default gemstone-rs.codegen-profiles.json
gemstone-rs codegen diff-profile default gemstone-rs.codegen-profiles.json
gemstone-rs codegen check-profile default gemstone-rs.codegen-profiles.json
gemstone-rs codegen explain-profile --json default gemstone-rs.codegen-profiles.json
gemstone-rs codegen generate-profile default gemstone-rs.codegen-profiles.json
```

See Codegen Profile Schema and `schemas/gemstone-rs.codegen-profiles.schema.json` for editor validation.

The project profile file uses this shape:

```
{"kind":"gemstone-rs-explorer-codegen-profiles","version":1,"profiles":
[{"name":"default","config":"examples/codegen/gemstone-
rs.codegen","root":"","mapped":"BookingDraft","className":"Object"}]}
```

Start the explorer with a project root:

```
gemstone-rs-explorer --port 8787 --codegen-root /path/to/gemstone-rs
```

Profile writes are disabled unless the explorer was started with `--allow-write`. The server rejects path traversal in `config=` and `profile_file=` after URL decoding, rejects traversal in `root=`, keeps writes under the configured codegen root by default, and requires `--allow-absolute-write-paths` before absolute write targets are accepted. Project profile saves reject unsupported top-level fields, unsupported profile fields, missing or duplicate profile names, and non-string profile values.

## Generated Shape

For:

```
method = Object>>printString | return=String | doc=Return the receiver printString.
```

The generated wrapper includes:

```
pub fn print_string(&mut self) -> Result<String> {
    let value = self.session.perform(self.oop, "printString", &[])?;
    match value {
        Value::String(value) => Ok(value),
        Value::Oop(oop) => self.session.fetch_string(oop),
        other => Err(Error::UnexpectedType {
            expected: "String",
            actual: format!("{other:?}"),
        }),
    }
}
```

Generated files are meant to be checked in. That makes diffs, reviews, and editor indexing predictable.

## Generated Wrapper App

The repository includes a small live example that imports the checked-in generated wrapper and calls `Object>>printString` on a small integer:

```
cargo run -p gemstone-rs --example generated_wrapper_app
```

Expected output:

```
generated wrapper printString: 7
```

The example uses the generated `Object::from_oop` constructor:

```
#[path = "../../../examples/codegen/generated/gemstone_wrappers.rs"]
mod gemstone_wrappers;

use gemstone_rs::{Config, Session};

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env()?)?;
    let oop = session.smallint_oop(7);
    let mut object = gemstone_wrappers::Object::from_oop(&mut session, oop);
    println!("{}", object.print_string()?);
    Ok(())
}
```

# Generated Object Mapping

The generated file can also include Rust data structs that implement `BridgeMapped`. Run:

```
cargo run -p gemstone-rs --example generated_mapping_app
```

Expected output:

```
generated mapped payload: BookingDraft { name: "Tariq", amount: 100, currency: "GBP",
tags: ["priority", "demo"], labels: {"source": "generated"}, note: Some("window
seat") }
```

The config-driven mapping emits a struct like:

```
#[derive(Clone, Debug, Eq, PartialEq)]
pub struct BookingDraft {
    pub name: String,
    pub amount: i64,
    pub currency: String,
    pub tags: Vec<String>,
    /// String-keyed labels.
    pub labels: BTreeMap<String, String>,
    /// Optional note.
    pub note: Option<String>,
}
```

and an implementation of `BridgeMapped` so it works with:

```
bridge_root.put_mapped("GeneratedBookingDraft", &draft)?;
let loaded: BookingDraft = bridge_root.get_mapped("GeneratedBookingDraft")?;
```

## VS Code Workflow

The `gemstone-rs Workbench` extension exposes the same flow in the GemStone RS sidebar:

- Discover from Live Stone
- Generate Mapping Config
- Preview BridgeRoot
- Run Generated Mapping Example
- Preview Wrappers
- Diff Generated Output
- Check Freshness
- Generate Wrappers
- Load Project Profiles
- Save Project Profiles
- Export Codegen Profile
- Validate Project Profiles
- Check Project Profiles
- Open Codegen Docs

`Generate Wrappers` shows the diff first and asks before writing when output would change.

> This PDF was generated locally from `docs/`. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.